Olimpiada AI

Algorytmy Regresji Liniowej i Wielomianowej Support Vector Machines

Wykład przygotowujący do Olimpiady Sztucznej Inteligencji

10.10.2026

Jakub Susoł

Mówimy o regresjach, klasyfikacjach...
Ale czym to w ogóle jest?



Regresja a klasyfikacja

Regresja

Regresja jest określeniem zbioru metod, których celem jest przewidywanie wartości liczbowych na podstawie zależności zadanych parametrów

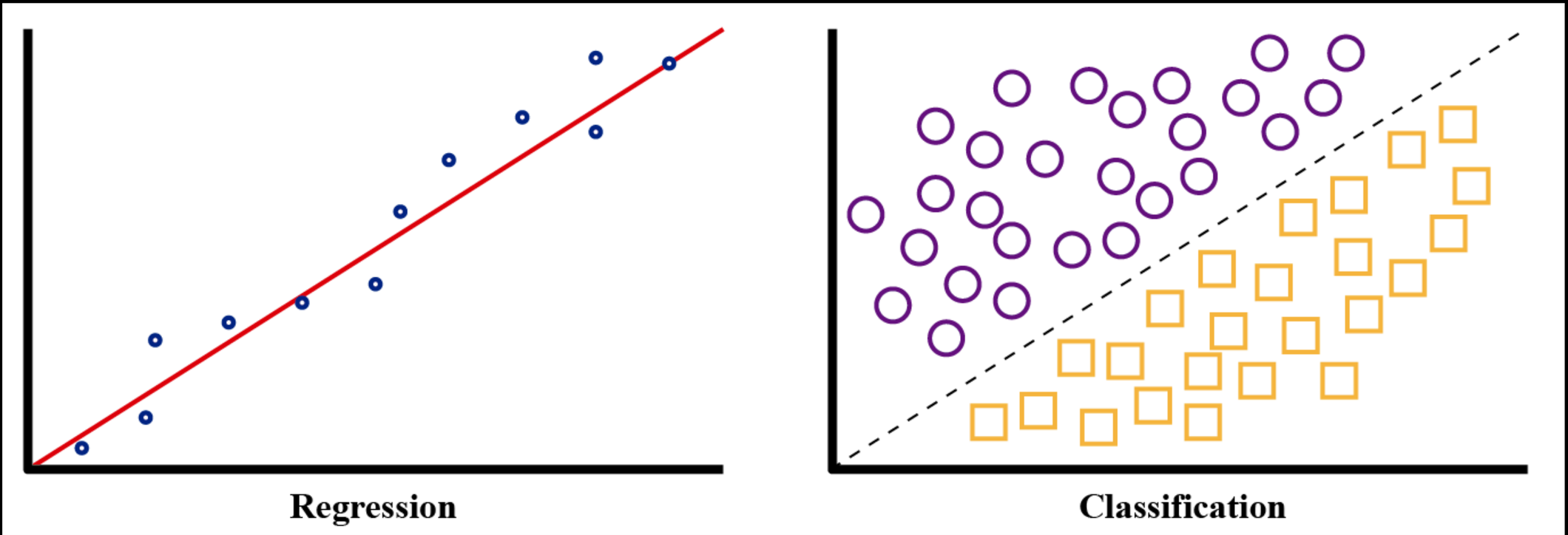
Albo po Polsku:
Wchodzi numer (lub zestaw numerów), wychodzi inny numer (lub zestaw numerów)

Klasyfikacja

Klasyfikacja jest określeniem zbioru metod, których celem jest zdefiniowanie przynależności obiektów do pewnego rodzaju grupy, bazując na zadanych parametrach

Albo po Polsku:
Wchodzi zestaw numerów, wychodzi paragon mówiący “Fajnie, jesteś w grupie A”

Regresja a klasyfikacja



https://panamahitek.com/en/what-is-the-difference-between-regression-and-classification-in-machine-learning/

Cytując klasyka...



https://joemonster.org/p/2130084

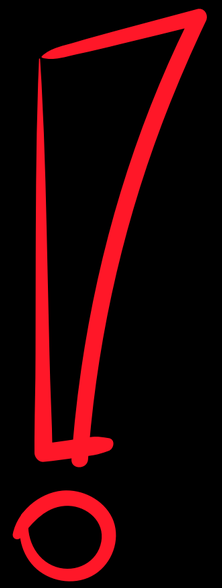
Otóż załóżmy nastepujące porównania:

- Cena mieszkania od powierzchni
- Wzrost dziecka od wieku
- Sprzedaż lodów od temperatury
- Ilość przejętych kont Discord od ilości wycieków danych



Czy znamy jeszcze jakieś przykłady takich zależności?

Takie zależności możemy przewidzieć właśnie za pomocą regresji!



Tylko jak to zrobić?

Najprostrzym przykładem regresji jest **Regresja Liniowa** (**ang.: Linear Regression**)

Bazuje ona na założeniu prostej definiowanej:

$$y = ax + b$$

Gdzie:
a - współczynnik nachylenia
b - wyraz wolny, punkt przecięcia z osią Y

Regresja Liniowa

Używając wzoru **y = *a*x + b,** będzimy próbować dobrać takie **a** i **b,** żeby prosta przecinała zbiorowikso punktów danych z jak najmniejszym **błędem**.

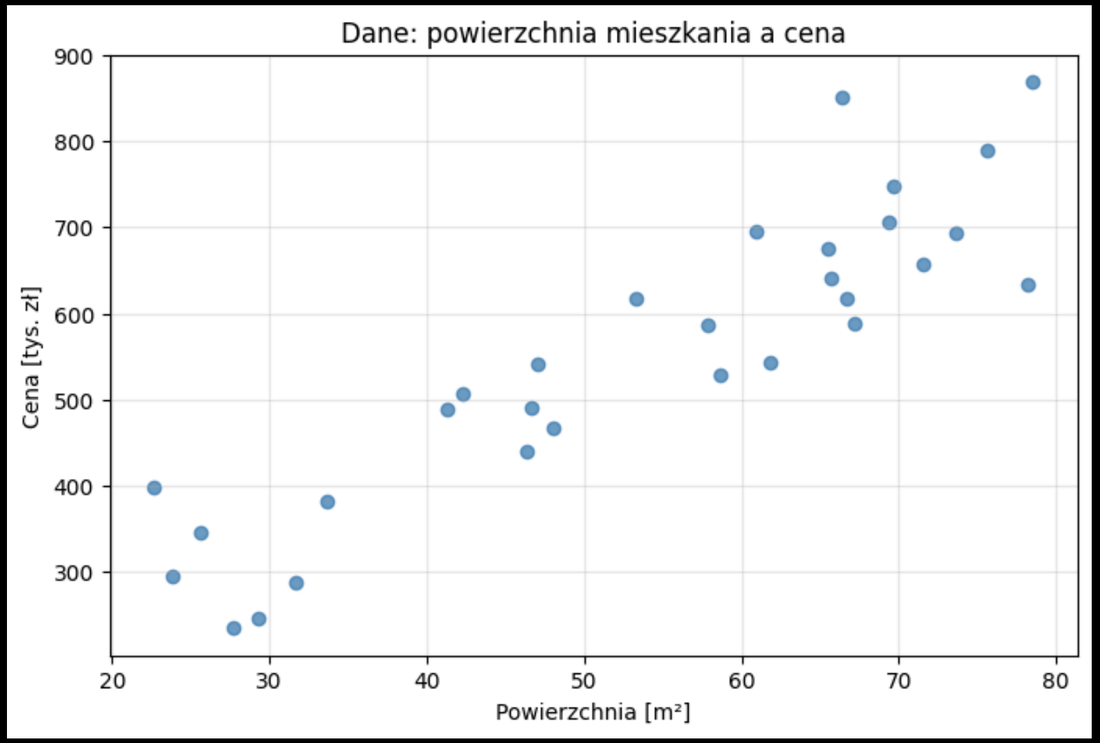
Ale hola hola, matma i **błędy**? Czy to może mieć jakikolwiek sens?

Ano i może! Nawet powinno!

Regresja Liniowa: o co chodzi?

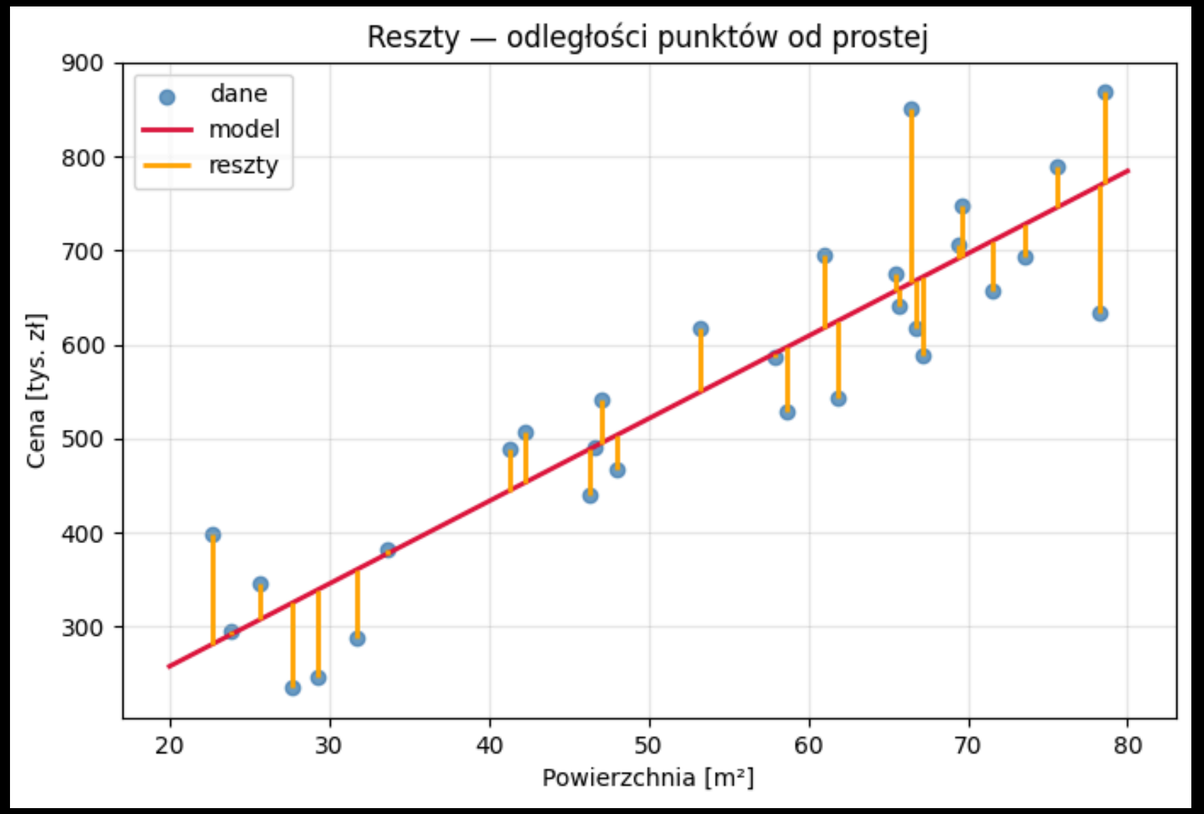
Mówiąc o regresji liniowej nie możemy zakładać braku błędu (ani w zasadzie w żadnym algorytmie przewidującym)!

Dane zwykle bywają srogo rozproszone, a my próbujemy tylko zgeneralizować zależności między nimi, aby wyłonić pewien schemat. Spójrzmy się na wspomniany wcześniej przykład:



Więc czym jest ten błąd?

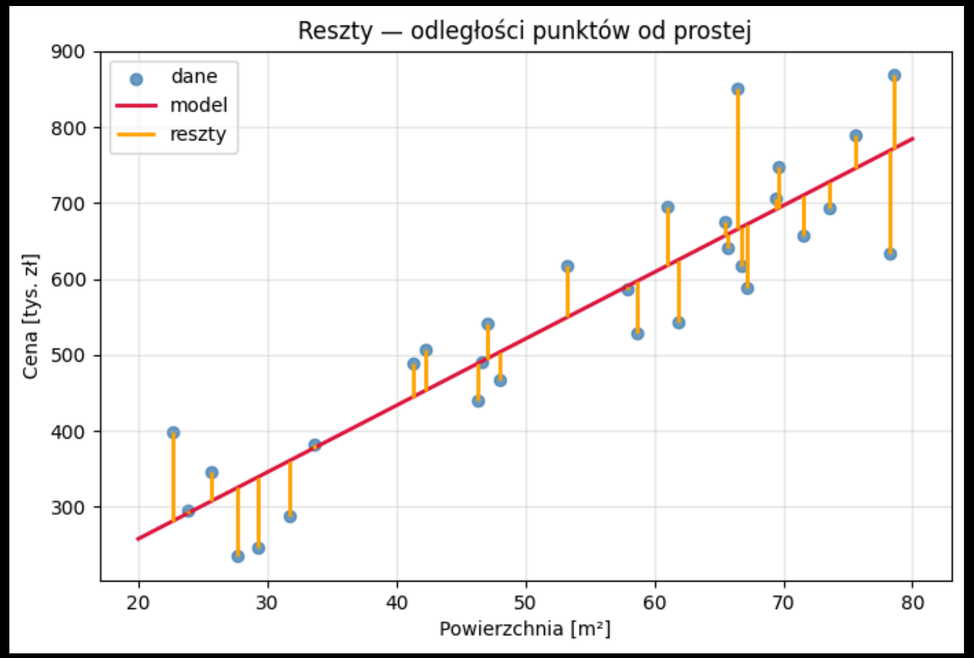
Błędem (**nazywanym też czasem resztą**) w tym wypadku, zakładając że poprowadzimy sobie losową prostą przez ten zbiór punktów będzie **kwadrat odległości punktu od krzywej!**



Metoda Najmniejszych Kwadratów

Takie podejście do optymalizacji dopasowania prostej do danych nazywamy **Metodą Najmniejszych Kwadratów.**

Jest ona banalna, bo prosta jest dobrze dopasowana tylko wtedy, kiedy **suma wszystkich błędów** (**kwadratów dystansów punktów od prostej**) **jest NAJMNIEJSZA.**



Algorytm Regresji Liniowej

Całościowo, algorytm na regresje liniową jest prosty:

1. Narysuj punkty na wykresie
2. Prowadź dowolną prostą przez chmurę punktów (próbujemy: y = ax + b)
3. Dla każdego punktu zmierz pionową odległość od prostej — to reszta
4. Podnieś każdą odległość do kwadratu (żeby minusy nie kasowały plusów)
5. Zsumuj wszystkie kwadraty — ta suma to "ocena" prostej (im mniejsza, tym lepiej)
6. Popraw lekko a i b i policz sumę od nowa
7. Powtarzaj, aż suma przestanie się zmniejszać — znaleziona prosta to nasz model

A jak ocenić miarodajność wyników?

Aby mieć pewność, że model przypadkiem nie halucynuje, musimy ocenić jego dopasowanie.

Najszerzej używaną miarą do ocany dopasowania jest współczynnik R^2.

$$\mathbf{R^2 = 1 - sum((y - y_pred)^2) / sum((y - y_średnia)^2)}$$

To jest wzór generalny, ale zwykle to się liczy funkcją r2_score :D

Patrzy się na to w następujący sposób:

Im bliżej współczynnik R^2 jest 1, tym lepiej model opisuje zależność danych.

Gdzie jest haczyk?



Problemy Regresji Liniowej

Wszystko wygląda fajnie, algorytm jest prosty i szybko się liczy, więc musi być coś, czego nikt nie mówi żeby nie zniechęcić programisty...

Głównymi problemami Regresji Liniowej są:

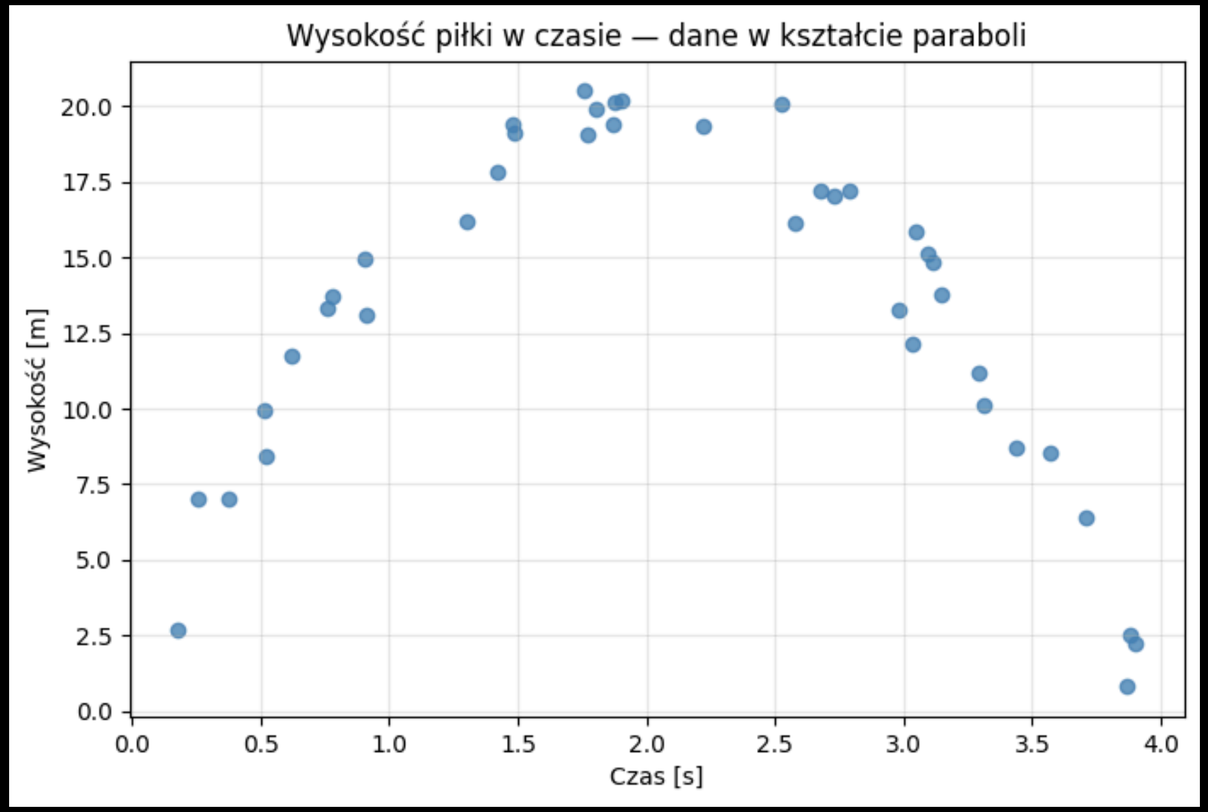
- Ekstrapolacja: przewidywanie daleko poza danymi bywa bezsensowne
- Wartości odstające: jeden dziwny punkt potrafi "przechylić" prostą
- Korelacja ≠ przyczynowość (przykład: ilość zjedzonych lodów a utonięcia)

A co jeśli zależność NIE jest liniowa?

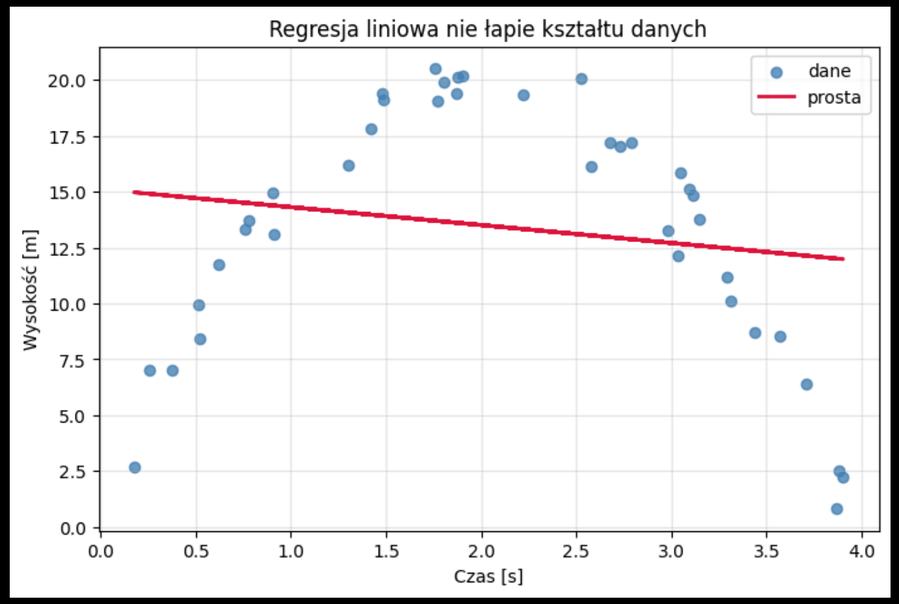


Załóżmy sobie nowy problem...

Rzucamy piłką w górę przed siebie i mierzymy jej wysokość i czas lotu do upadku. Powtarzamy to kilka razy i widzimy, że nasz wykres punktów przypomina terror każdego ucznia na matematyce: parabolę.



Regresja Wielomianowa

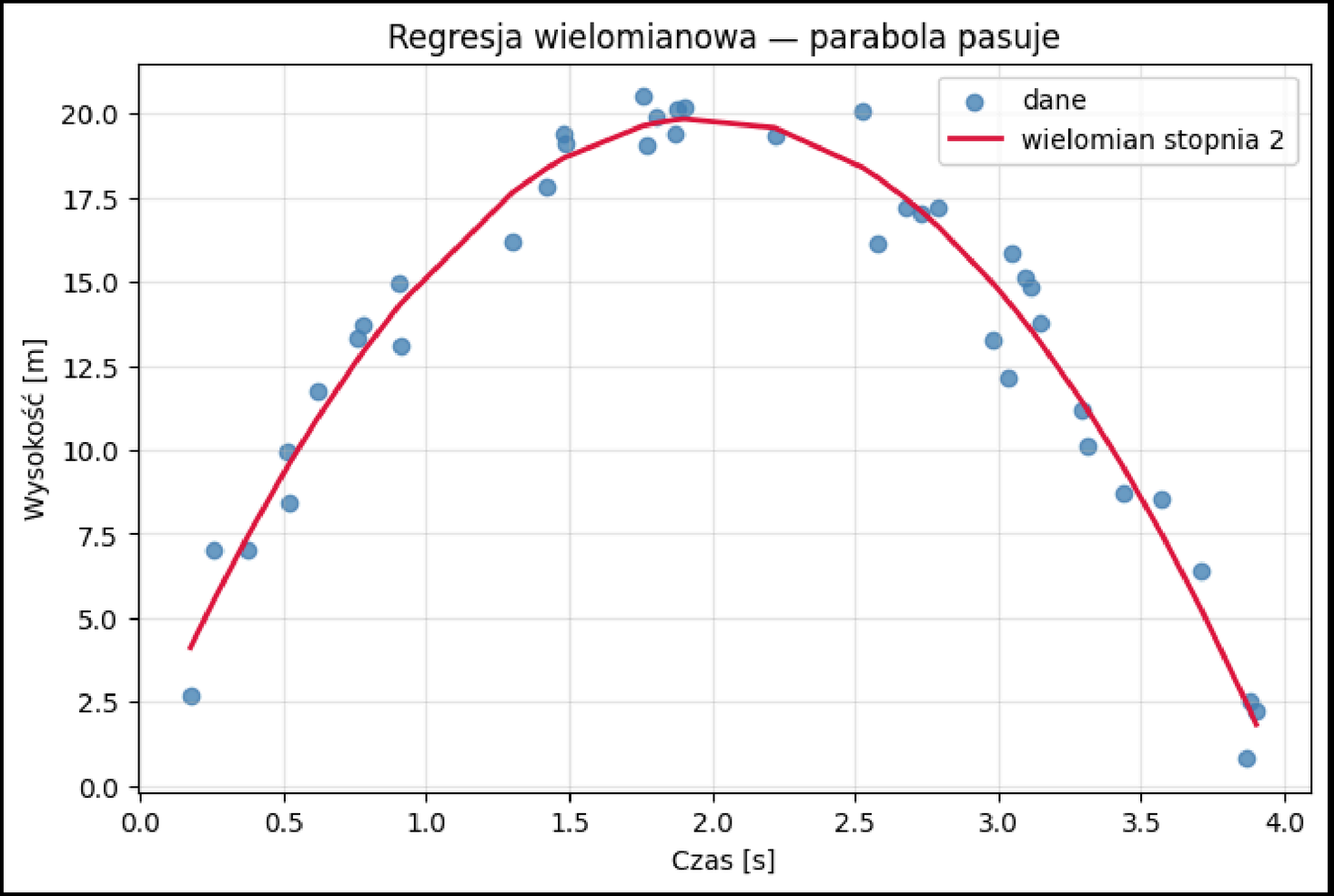


Takiego typu problemu, dość logicznie, nie rozwiążemy stawiając prostą, a z lekcji matematyki wiemy, że istnieją funkcje wielomianowe, a przykładem takiej funkcji jest właśnie nasza parabola:

$$y = ax^2 + bx + c$$

W takim razie postanawiamy sobie użyć takiej funkcji żeby dopasować model i...

VOILA!


Regresja wielomianowa — parabola pasuje
dane
wielomian stopnia 2
Wysokość [m]
Czas [s]
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

Regresja Wielomianowa

Sama Regresja **Wielomianowa,** którą teraz zastosowaliśmy nie ogranicza się jedynie do funkcji kwadratowych. Możemy zadać jej dowolny stopień wielomianowości, a pod maską jest to interpretowane jako zadanie wzoru:

$$y = a_0 + a_1x + a_2x^2 + a_3x^3 + \text{...itd}$$

Tutaj operujemy tylko współczynnikami (a), a nasze dane wejściowe (x) są "automagicznie" transformowane do odpowiedniej formy.

Przykłady odnośnie regresji wielomianowej głębiej pokazane są w kodzie, a sam algorytm używa tej samej logiki co Regresja Liniowa.

Problemy Regresji Wielomianowej

Poza wspomnianymi już problemami Regresji Liniowej:

- Ekstrapolacja: przewidywanie daleko poza danymi bywa bezsensowne
- Wartości odstające: jeden dziwny punkt potrafi "przechylić" prostą
- Korelacja ≠ przyczynowość (przykład: ilość zjedzonych lodów a utonięcia)

Musimy też zwracać uwagę na:

- Niedouczenie: za prosty model (prosta przez dane w kształcie fali)
- Przeuczenie: za skomplikowany model (wężyk przez każdy punkt)

Odpowiedzieliśmy sobie na pytanie “A dlaczego?”



https://joemonster.org/p/2130084

Więc teraz czas na odpowiedź do pytania “A komu to potrzebne?”



https://makeameme.org/meme/logistic-regression-is-7affe15c96

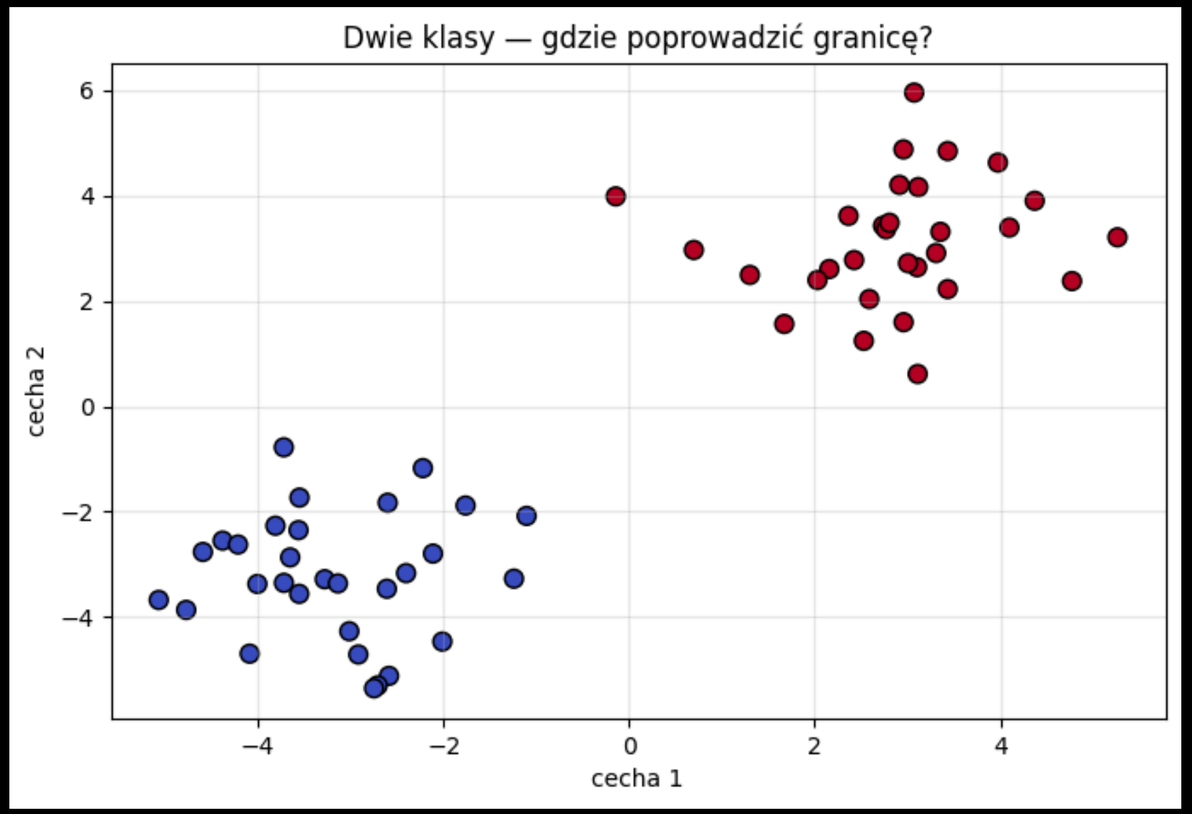
Regresja w klasyfikatorach

Algorytmów regresji możemy użyć też, aby poprowadzić krzywą oddzielającą zbiorowisko punktów w taki sposób, żeby można było je łatwo zkategoryzować do pewnej grupy.

Jednym z najszerzej używanych do tego podejść jest algorytm SVM - Support Vector Machines, albo po naszemu Maszyna Wektorów Nośnych

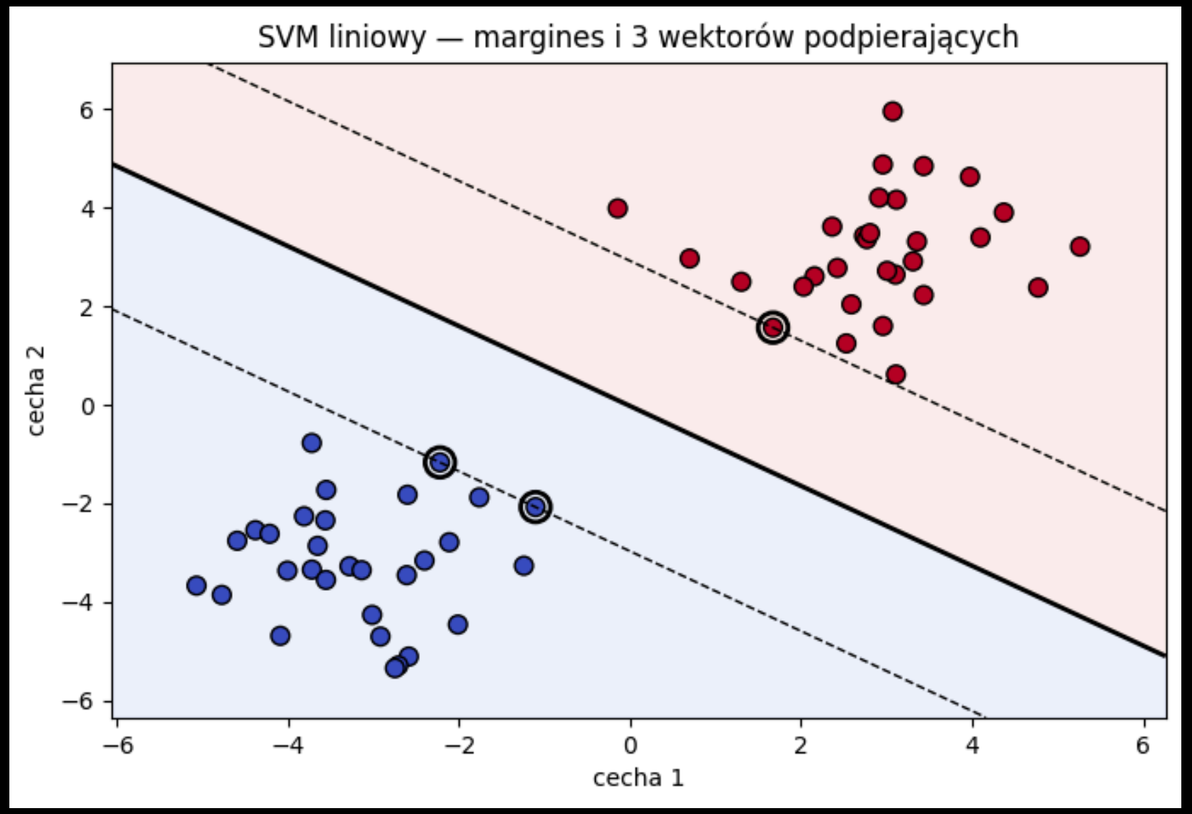
SVM

Załóżmy sobie że mamy dane pochodzące z dwóch miejsc w Polsce i wykreśliliśmy je sobie na wykresie



SVM

Algorytm SVM ma na zadanie poprowadzić krzywą oddzielającą te zgrupowania punktów tak, aby przy dodaniu nowych danych można było je od razy zklasyfikować, zakładając też prawdopodobieństwo przynależności pomiaru do danej klasy patrząc na odległość punktu od **marginesu**

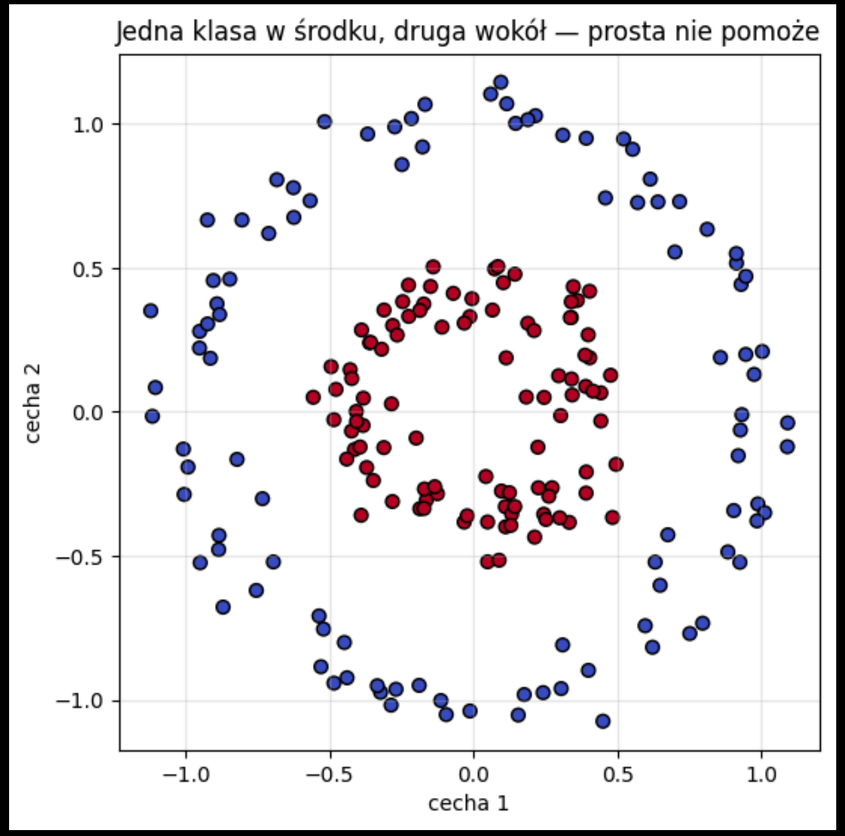


Algorytm SVM

1. Narysuj punkty obu klas na płaszczyźnie
2. Prowadź dowolną prostą, która poprawnie rozdziela klasy (zero punktów "po złej stronie")
3. Znajdź punkty z każdej klasy najbliżej tej prostej, to kandydaci na wektory podpierające
4. Zmierz odległość między najbliższymi punktami obu klas
5. Przesuwaj i obracaj prostą tak, żeby margines był jak najszerszy (punkty w środku nie mają na to wpływu, liczą się tylko te najbliższe)
6. Gdy margines przestaje się poszerzać, koniec: granica decyzji to środek najszerszego marginersu

A co jeśli mamy kompletny brak liniowości?

Zakładamy nowy problem. Na stole rozsypaliśmy piłki, i ułożyły się w sposób taki, ze piłki niebieskie leżą na zewnątrz, a czerwone wewnątrz powierzchni stołu.



Różne podejścia w samym SVM

Tutaj algorytm SVM ma dodatkową własność, opiera się na **kernelu.**

Taki kernel to nic innego jak funkcja, pomagająca wykonać podział poprzez przeniesienie istniejących punktów do nowej przestrzeni, zwiększając lub zmniejszając wymiarowość danych.

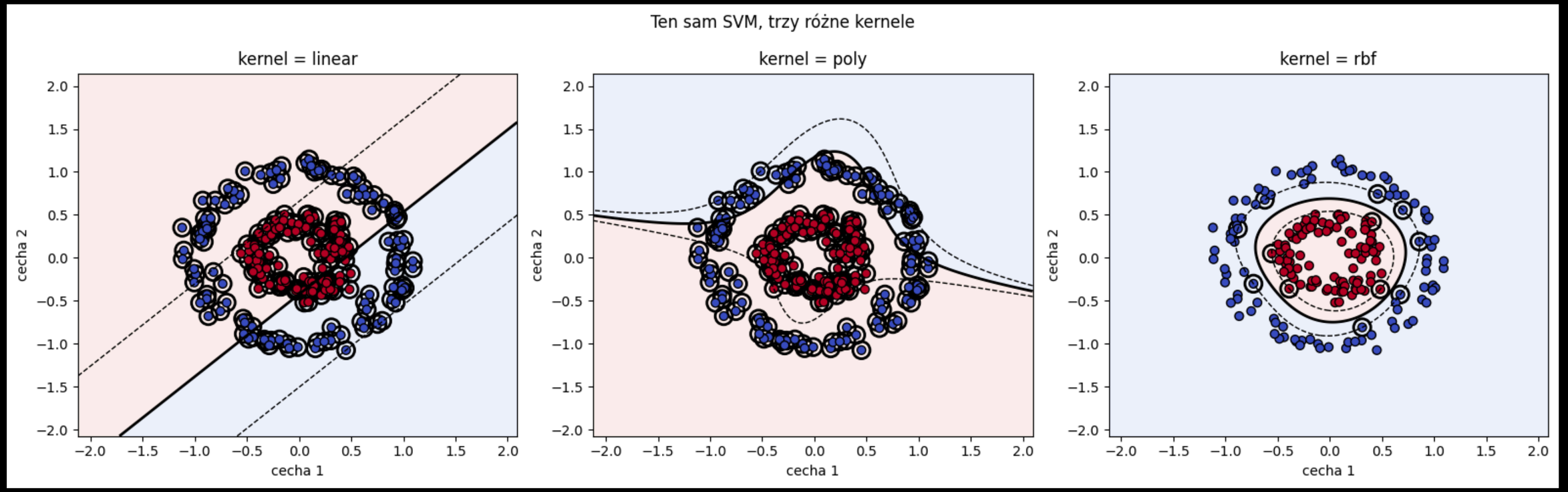
Co jest ważne to to, że takie podejście pozwala nam użyć też tzw. “kernel trick”, czyli użycia kernela do zmiejszenia ilości operacji wykonywanych przez komputer.

Różne podejścia w samym SVM

Mamy możliwosc klasyfikacji za pomocą różnych kerneli

- Liniowy: zwykła prosta granica (dane łatwo rozdzielne)
- Wielomianowy: granice zakrzywione (nawiązanie do regresji wielomianowej!)
- RBF (gaussowski): najpopularniejszy; granica może przybierać dowolnie złożone kształty

Różne podejścia w samym SVM



Kernel trick

- Podnoszenie do wysokich wymiarów jest kosztowne obliczeniowo
- Sztuczka: liczymy "podobieństwo" punktów tak, jakbyśmy byli w wysokim wymiarze, bez faktycznego liczenia tam współrzędnych
- Analogia: nie musisz budować wieżowca, żeby wiedzieć, jak ludzie na różnych piętrach są od siebie oddaleni, wystarczy "tabela odległości" (kernel)

1. Weź splątane dane, których nie rozdzieli żadna prosta
2. Wybierz kernel, np. RBF, funkcję, która dla dwóch punktów zwraca "jak podobne" (liczba od 0 do 1)
3. Policz podobieństwa punktów do siebie, to zamiennik "podniesienia do wysokiego wymiaru"
4. SVM szuka granicy tak jak zwykle, tylko operuje na podobieństwach
5. Efekt: z zewnątrz granica wygląda jak krzywa, a "w wyższym wymiarze" to nadal prosta
6. Nowy punkt klasyfikujemy, porównując go (kernelem) z wektorami podpierającymi

Pytania



Dziękuję za Uwagę!